

COMPETITIVE PROGRAMMING

SMTYON

C++ Template Library

A ready-to-print collection of competitive programming templates — data structures, graph algorithms, number theory, string algorithms, dynamic programming and more.

52 templates · **7** sections · C++17

Prepared for printing · generated from the SMTYON repository

Table of Contents

Core (2)

Matrix expo	4
template	5

DP (2)

CHT	6
SOS	7

Data Structures (16)

2D Fenwick	9
2D sparse table	9
Binary Trie	11
DSU	12
DSU rollback	12
Fenwick	14
Implicit segment	14
IterativeSegTree	16
MO with DSU	17
Mo	20
Persistent segment	20
Segtree	22
Segtree lazy	23
Sparse table	25
Sqrt Decomposition	26
String Trie	27

Graph (7)

Bridges & Articulation Points	28
Floyd Warshall	30
LCA	31
SCC	31
Sack	33
matching	34
small to large	36

Math (13)

Counting	37
----------------	----

Expected Value	38
BigInt	39
FFT	40
FFT mod	40
FWHT	42
NTT	43
poly pow	46
string matching	46
Laws	47
Math	47
Prime factorization	52
geometric series	52

Strings (6)

2D Hash	53
Hashing	54
Kmp	56
Manacher	56
Suffix Array	57
Z Algo	60

Other (6)

Count Inversions	61
Multiset Hashing	62
Ordered set	63
Pragma	63
Random	63
Super FIO	63

Page numbers refer to the printed document.

Core (2)

Root

Matrix expo

Matrix_expo.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(),a.end()
#define ll long long
#define int long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7 , inf = 1e9 , MAXN = 1e5+5;
#define matrix vector<vector<ll>>
#define zero(n,m) matrix(n , vector<ll>(m,0))

void init(matrix &M , matrix &res)
{
    // initialize the values of the matrices
    M[0][0] = 0 , M[0][1] = 1 , M[1][0] = 1 , M[1][1] = 1;
    res[0][0]=0 , res[1][0]=1;
}

matrix mul(matrix &a , matrix &b)
{
    int n = a.size() , m = b[0].size();
    matrix res = zero(n , m);

    for(int i=0; i<n; i++)
    {
        for(int j=0; j<m; j++)
        {
            for(int k=0; k<n; k++)
            {
                res[i][j] += a[i][k]*b[k][j];
                res[i][j] %= mod;
            }
        }
    }
    return res;
}

matrix matrix_fp(matrix &a , ll p)
{
    if (p==1) return a;
    matrix res = matrix_fp(a , p/2);
    res = mul(res , res);
    if (p%2) res = mul(res , a);

    return res;
}

int main()
{
    SMTYON
    ll n;
    cin >> n;
    if (n<2)
    {
        cout << n << "\n";
    }
    else
    {
        matrix M = zero(2,3) , A = zero(2,1);
        init(M , A);

        M = matrix_fp(M , n-2+1);
    }
}

```

```

        A = mul(M , A);

        cout << A[1][0] << '\n';
    }

    return 0;
}

// if you have to matrices depend on i
matrix power(matrix a, ll p) {
    int n = a.size();
    matrix res = zero(n, n);
    for(int i = 0; i < n; i++) res[i][i] = 1;

    while(p > 0) {
        if(p & 1) res = mul(res, a);
        a = mul(a, a);
        p >>= 1;
    }
    return res;
}

matrix matrix_fp(matrix &o , matrix &e , ll p , int type)// o -> odd
{
    if (p == 0) {
        int n = o.size();
        matrix res = zero(n, n);
        for(int i=0; i<n; i++) res[i][i] = 1;
        return res;
    }

    matrix base;
    if (type == 0) base = mul(o, e);
    else          base = mul(e, o);

    matrix res = power(base, p / 2);
    if (p % 2 == 1) {
        res = mul(res, (type == 0) ? o : e);
    }

    return res;
}

```

template

template.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a)  a.begin(),a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7 , inf = 1e9 , MAXN = 1e5+5;

signed main() {
    SMTYON
    int x_x = 1;
    //cin >> x_x;
    while (x_x--) {

    }

    return 0;
}

```

DP (2)

DP

CHT

DP/CHT.cpp

```

#include <deque>
#include <iostream>
#include <vector>
using namespace std;

/*
Finding the minimum value

Note: If you need to find the maximum instead of the minimum,
you can simply add your lines as (-m, -c)
and then negate the result of the query
*/

// Represents a line y = m*x + c
struct Line {
    long long m, c;

    // Evaluates the line equation at a given x
    long long eval(long long x) { return m * x + c; }
};

struct ConvexHullTrick {
    deque<Line> dq;

    // Helper to determine if line l3 makes line l2 redundant.
    // Given lines l1, l2, l3 in decreasing order of slopes:
    // l2 becomes useless if the intersection of l1 and l3 is
    // to the left of the intersection of l1 and l2.
    // Mathematical condition: (c3 - c1) / (m1 - m3) <= (c2 - c1) / (m1 - m2)
    // To avoid floating point division issues, we cross-multiply.
    bool useless(Line l1, Line l2, Line l3) {
        // (c3 - c1) * (m1 - m2) <= (c2 - c1) * (m1 - m3)
        // Note: double can be used if coordinates are extremely large,
        // but long long is safer for precision if it fits without overflow.
        return (double)(l3.c - l1.c) * (l1.m - l2.m) <=
            (double)(l2.c - l1.c) * (l1.m - l3.m);
    }

    // Add a new line y = m*x + c
    // REQUIRED: Slopes 'm' MUST BE ADDED IN MONOTONICALLY DECREASING ORDER.
    void add(long long m, long long c) {
        Line l3 = {m, c};

        // As long as there are at least two lines in the deque, check if the
        // new line makes the last line in the envelope redundant.
        while (dq.size() >= 2 && useless(dq[dq.size() - 2], dq.back(), l3)) {
            dq.pop_back(); // Remove redundant line
        }
        dq.push_back(l3);
    }

    // Find the minimum value among all lines at a given x
    // REQUIRED: Query values 'x' MUST BE MONOTONICALLY INCREASING.
    long long query(long long x) {
        // As x increases, the optimal line might shift to the right.
        // We evaluate the first two lines. If the second line is better (gives a
        // lower y), the first line will never be optimal again (due to increasing
        // x), so we drop it.
        while (dq.size() >= 2 && dq[0].eval(x) >= dq[1].eval(x)) {
            dq.pop_front();
        }

        // The first line in the deque is now the optimal line for this x.
        return dq.front().eval(x);
    }
};

```

```

    }
};

int main() {
    ConvexHullTrick cht;

    // Example Usage:
    // Adding lines y = mx + c (must be in decreasing order of m)
    cht.add(10, 5);
    cht.add(8, 12);
    cht.add(5, 20);

    // Querying minimum at x (must be in increasing order of x)
    cout << "Min at x=2: " << cht.query(2) << endl;
    cout << "Min at x=5: " << cht.query(5) << endl;

    return 0;
}

```

SOS

DP/SOS.cpp

```

// sum over subset.

/*
O(3^N):
for (int mask = 0; mask < (1 << N); mask++) {
    for (int submask = mask; submask > 0; submask = (submask - 1) & mask) {
        F[mask] += A[submask];
    }
    F[mask] += A[0];
}
*/

// SOS DP:

// 1. Without Memory Optimization (2D Array)

/* This implementation directly translates the mathematical recurrence relation into code.
We use a 2D array dp[mask][i], which represents the sum of subsets for a given mask where
we are only allowed to toggle the first i bits (from the right).

Time Complexity: O(N * 2^N)
Space Complexity: O(N * 2^N)
*/
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int N = 20; // Example: 20 bits
    int MAX_MASK = 1 << N;

    vector<int> A(MAX_MASK, 1); // Example array initialized with 1s

    // dp[mask][i]
    // mask goes up to 2^N, i goes up to N
    vector<vector<int>> dp(MAX_MASK, vector<int>(N + 1, 0));

    // Base Case: i = 0 (No bits are allowed to differ)
    // The only valid subset is the mask itself.
    for (int mask = 0; mask < MAX_MASK; mask++) {
        dp[mask][0] = A[mask];
    }

    // DP Transitions
    for (int i = 1; i <= N; i++) {
        for (int mask = 0; mask < MAX_MASK; mask++) {
            // Check if the (i-1)-th bit is ON
            if (mask & (1 << (i - 1))) {
                // If ON: Valid subsets can have this bit as 1 or 0.
            }
        }
    }
}

```

```

        // Add the state where the bit stays 1, and the state where it toggles to 0.
        dp[mask][i] = dp[mask][i - 1] + dp[mask ^ (1 << (i - 1))][i - 1];
    } else {
        // If OFF: Valid subsets must also have this bit as 0.
        dp[mask][i] = dp[mask][i - 1];
    }
}

// The final answer for any mask is stored in dp[mask][N]
// Example output for mask 3 (binary 00...011, which has 4 subsets: 0, 1, 2, 3)
cout << "Sum for mask 3: " << dp[3][N] << "\n";

return 0;
}

// 2. With Memory Optimization (1D Array) In the 2D version
/*
calculating dp[mask][i] only requires values from the previous step, dp[...][i-1].
Because of how bitwise operations work, when we update the state for an active bit 1,
we are reading from the state where that bit is 0 (mask ^ (1 << i)).
Since we only ever read from states with a 0 at the current bit position
to update states with a 1 at the current bit position,
we can safely overwrite the array in place without corrupting the data needed for the current step.

Time Complexity:  $O(N * 2^N)$ 
Space Complexity:  $O(2^N)$ 
*/
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int N = 20; // Example: 20 bits
    int MAX_MASK = 1 << N;

    vector<int> A(MAX_MASK, 1); // Example array initialized with 1s

    // We only need a 1D array of size  $2^N$ 
    vector<int> F(MAX_MASK);

    // Step 1: Base Case initialization
    for (int mask = 0; mask < MAX_MASK; ++mask) {
        F[mask] = A[mask];
    }

    // Step 2: In-place SOS DP
    for (int i = 0; i < N; ++i) {
        for (int mask = 0; mask < MAX_MASK; ++mask) {
            // If the i-th bit of the mask is ON
            if (mask & (1 << i)) {
                // Add the value of the subset where the i-th bit is OFF
                F[mask] += F[mask ^ (1 << i)];
            }
        }
    }

    // The final answer for any mask is directly stored in F[mask]
    // Example output for mask 3 (binary 00...011, which has 4 subsets: 0, 1, 2, 3)
    cout << "Sum for mask 3: " << F[3] << "\n";

    return 0;
}

```


Data Structures (16)

DS

2D Fenwick

DS/2D Fenwick.cpp

```

struct FenwickTree2D { // 1-based
    int n, m;
    vector<vector<int>> tree;

    FenwickTree2D(int n_size, int m_size) {
        n = n_size;
        m = m_size;
        tree.assign(n+1, vector<int>(m+1, 0));
    }

    // Point update: Add val to coordinate (x, y)
    void update(int x, int y, int val) {
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) {
                tree[i][j] += val;
            }
        }
    }

    // Prefix query: Sum of rectangle from (1, 1) to (x, y)
    int query(int x, int y) {
        int sum = 0;
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                sum += tree[i][j];
            }
        }
        return sum;
    }

    int query_range(int x1, int y1, int x2, int y2) {
        return query(x2, y2)
            - query(x1 - 1, y2)
            - query(x2, y1 - 1)
            + query(x1 - 1, y1 - 1);
    }
};

```

2D sparse table

DS/2D sparse table.cpp

```

const int MAXN = 505; // Max Rows
const int MAXM = 505; // Max Columns
const int LOGN = 10; // log2(MAXN) + 1
const int LOGM = 10; // log2(MAXM) + 1

class SparseTable2D {
private:
    // table[jr][ir][jc][ic]
    int st[LOGN][MAXN][LOGM][MAXM];
    int log2_val[max(MAXN, MAXM) + 1];
    int n, m;

    void precomputeLogs() {
        log2_val[1] = 0;
        int max_dim = max(n, m);
        for (int i = 2; i <= max_dim; i++) {
            log2_val[i] = log2_val[i / 2] + 1;
        }
    }

public:

```

```

void build(const vector<vector<int>>& matrix) {
    if (matrix.empty() || matrix[0].empty()) return;
    n = matrix.size();
    m = matrix[0].size();

    precomputeLogs();

    // 1. Build the 1D Sparse Table for each row (jr = 0)
    for (int ir = 0; ir < n; ir++) {
        // Base case
        for (int ic = 0; ic < m; ic++) {
            st[0][ir][0][ic] = matrix[ir][ic];
        }
        // Fill 1D RMQ for the current row
        for (int jc = 1; jc <= log2_val[m]; jc++) {
            for (int ic = 0; ic + (1 << (jc - 1)) < m; ic++) {
                st[0][ir][jc][ic] = min(st[0][ir][jc - 1][ic],
                                         st[0][ir][jc - 1][ic + (1 << (jc - 1))]);
            }
        }
    }

    // 2. Build the 2D Sparse Table using the 1D tables as a base
    for (int jr = 1; jr <= log2_val[n]; jr++) {
        for (int ir = 0; ir + (1 << (jr - 1)) < n; ir++) {
            for (int jc = 0; jc <= log2_val[m]; jc++) {
                for (int ic = 0; ic < m; ic++) {
                    // We take the minimum of the top half and bottom half
                    st[jr][ir][jc][ic] = min(st[jr - 1][ir][jc][ic],
                                              st[jr - 1][ir + (1 << (jr - 1))][jc][ic]);
                }
            }
        }
    }

    // Query in O(1)
    int query(int x1, int y1, int x2, int y2) {
        int lenx = x2 - x1 + 1;
        int kx = log2_val[lenx];

        int leny = y2 - y1 + 1;
        int ky = log2_val[leny];

        int min_R1 = min(st[kx][x1][ky][y1],
                        st[kx][x1][ky][y2 + 1 - (1 << ky)]);

        int min_R2 = min(st[kx][x2 + 1 - (1 << kx)][ky][y1],
                        st[kx][x2 + 1 - (1 << kx)][ky][y2 + 1 - (1 << ky)]);

        return min(min_R1, min_R2);
    }
};

```

Binary Trie

DS/Binary_Trie.cpp

```
struct BinaryTrie {
    const int BIT = 31;
    struct Node {
        int ch[2]{};
        int p = 0;
    };
    vector<Node> tree;
    BinaryTrie(){
        tree.emplace_back();
        insert(0);
    }

    void insert(int x) {
        int node = 0;
        for (int i = BIT; ~i; --i) {
            int bit = (1 << i);
            bool cur = (x & bit);
            if (tree[node].ch[cur] == 0) {
                tree[node].ch[cur] = tree.size();
                tree.emplace_back();
            }
            node = tree[node].ch[cur];
            tree[node].p++;
        }
    }

    void erase(int x) {
        int node = 0;
        for (int i = BIT; ~i; --i) {
            int bit = (1 << i);
            bool cur = (x & bit);
            node = tree[node].ch[cur];
            tree[node].p--;
        }
    }

    int mx_xor(int x) {
        int node = 0;
        int ret = 0;
        for (int i = BIT; ~i; --i) {
            int bit = (1 << i);
            bool cur = (x & bit);
            if (tree[tree[node].ch[cur ^ 1]].p) {
                ret += (1 << i);
                cur ^= 1;
            }
            node = tree[node].ch[cur];
        }
        return ret;
    }
};
```

DSU

DS/DSU.cpp

```

class DisjointSets {
private:
    vector<int> parents;
    vector<int> sizes;

public:
    DisjointSets(int size) : parents(size), sizes(size, 1) {
        for (int i = 0; i < size; i++) {
            parents[i] = i;
        }
    }

    /** @return the "representative" node in x's component */
    int find(int x) {
        return parents[x] == x ? x : (parents[x] = find(parents[x]));
    }

    /** @return whether the merge changed connectivity */
    bool unite(int x, int y) {
        int x_root = find(x);
        int y_root = find(y);
        if (x_root == y_root) {
            return false;
        }

        if (sizes[x_root] < sizes[y_root]) {
            swap(x_root, y_root);
        }
        sizes[x_root] += sizes[y_root];
        parents[y_root] = x_root;
        return true;
    }

    /** @return whether x and y are in the same connected component */
    bool connected(int x, int y) { return find(x) == find(y); }
};

```

DSU rollback

DS/DSU_rollback.cpp

```

struct DSU {
    vector<int> par, sz;
    struct State {
        int u, v; // u becomes child of v
        bool united;
    };
    vector<State> history;

    DSU(int n) {
        par.resize(n);
        iota(all(par), 0);
        sz.assign(n, 1);
    }

    int find(int i) {
        while (par[i] != i)
            i = par[i];
        return i;
    }

    bool unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);
        if (root_i != root_j) {
            if (sz[root_i] < sz[root_j])
                swap(root_i, root_j);

```

```
// Attach root_j to root_i
history.push_back({root_j, root_i, true});
par[root_j] = root_i;
sz[root_i] += sz[root_j];
return true;
}
history.push_back({-1, -1, false}); // No merge happened
return false;
}

void rollback() {
    if (history.empty())
        return;
    State last = history.back();
    history.pop_back();
    if (last.united) {
        // last.u was attached to last.v
        par[last.u] = last.u;
        sz[last.v] -= sz[last.u];
    }
}

int snapshot() { return history.size(); }

void rollbackTo(int snapshot) {
    while (history.size() > snapshot) {
        rollback();
    }
}

void reset(int n) {
    iota(par.begin(), par.end(), 0);
    sz.assign(n, 1);
    history.clear();
    comps = n;
}
};
```

Fenwick

DS/Fenwick.cpp

```

struct FenwickTree { // 1-based
    int n;
    vector<int> tree;

    FenwickTree(int size) {
        n = size;
        tree.assign(n + 1, 0);
    }

    // constructor from an existing array O(N)
    FenwickTree(const vector<int> &a) {
        FenwickTree(a.size());

        for (int i = 0; i < n; i++) {
            update(i + 1, a[i]);
        }
    }

    void update(int i, int val) {
        for (; i <= n; i += i & -i) {
            tree[i] += val;
        }
    }

    // Prefix sum: Sum of elements in [1, i]
    int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) {
            sum += tree[i];
        }
        return sum;
    }

    // Binary Lifting: Find the smallest 1-based index where prefix sum >= target
    // Assumes all frequencies are non-negative!
    int lower_bound(int target) {
        int pos = 0;

        // Find the greatest power of 2 <= n
        int bitMask = 1;
        while (bitMask <= n)
            bitMask <<= 1;
        bitMask >>= 1;

        for (; bitMask > 0; bitMask >>= 1) {
            if (pos + bitMask <= n && tree[pos + bitMask] < target) {
                pos += bitMask;
                target -= tree[pos];
            }
        }
        return pos + 1; // Return the 1-based index
    }
};

```

Implicit segment

DS/Implicit segment.cpp

```

#include <iostream>
#include <vector>

using namespace std;

struct Node {
    ll sum;
    int left_child;
    int right_child;
} NEUTRAL = {0, 0, 0};

```

```

struct ImplicitSegTree {
    ll MAX_R; // The maximum possible coordinate (e.g., 1e9)
    vector<Node> tree;
    int node_count;
    int root;

    // max_range is the upper bound of your array (e.g., 1e9 or 2e9)
    // expected_updates is usually Q (number of queries/updates)
    ImplicitSegTree(ll max_range, int expected_updates) {
        MAX_R = max_range;

        // Depth is roughly log2(max_range). For 1e9, it's ~30.
        // So each update creates at most ~32 nodes.
        int max_nodes = (expected_updates * 32) + 2;
        tree.resize(max_nodes);

        node_count = 1;
        root = create_node(); // Initialize the root node (index 1)
    }

    int create_node() {
        int id = node_count++;
        tree[id].sum = 0;
        tree[id].left_child = 0;
        tree[id].right_child = 0;
        return id;
    }

    Node merge(Node a, Node b) {
        Node res;
        res.sum = a.sum + b.sum;
        return res;
    }

    // Point update at index i (0-based)
    void update(ll i, ll val, int x, ll lx, ll rx) {
        if (rx - lx == 1) {
            tree[x].sum += val;
            return;
        }

        ll m = (rx + lx) / 2;

        if (i < m) {
            if (!tree[x].left_child)
                tree[x].left_child = create_node();
            update(i, val, tree[x].left_child, lx, m);
        } else {
            if (!tree[x].right_child)
                tree[x].right_child = create_node();
            update(i, val, tree[x].right_child, m, rx);
        }

        Node left_node = tree[x].left_child ? tree[tree[x].left_child] : NEUTRAL;
        Node right_node = tree[x].right_child ? tree[tree[x].right_child] : NEUTRAL;

        // IMPORTANT: Only update the sum. Do not overwrite tree[x] completely.
        tree[x].sum = merge(left_node, right_node).sum;
    }

    // Zero-based Range Query [l, r)
    Node query(ll l, ll r, int x, ll lx, ll rx) {
        if (lx >= r || rx <= l || x == 0)
            return NEUTRAL;
        if (lx >= l && rx <= r)
            return tree[x];

        ll m = (rx + lx) / 2;
        Node left_res = query(l, r, tree[x].left_child, lx, m);
        Node right_res = query(l, r, tree[x].right_child, m, rx);

        return merge(left_res, right_res);
    }
}

```

```

}

void update(ll i, ll val) { update(i, val, root, 0, MAX_R); }
ll query(ll l, ll r) { return query(l, r, root, 0, MAX_R).sum; }
};

int main() {
    int expected_updates = 100000;
    ll max_coordinate = 1000000000; // 1e9

    ImplicitSegTree st(max_coordinate, expected_updates);

    // Update coordinates that are extremely far apart
    st.update(5, 10);
    st.update(500000000, 20); // 5 * 10^8
    st.update(999999999, 30); // 10^9 - 1

    // Range queries over massive empty spaces are perfectly fast and safe
    cout << "Sum [0, 10): " << st.query(0, 10) << "\n"; // 10
    cout << "Sum [0, 600000000): " << st.query(0, 600000000) << "\n"; // 30
    cout << "Sum [0, 1000000000): " << st.query(0, 1000000000) << "\n"; // 60

    return 0;
}

```

IterativeSegTree

DS/IterativeSegTree.cpp

```

#include <bits/stdc++.h>
using namespace std;

template <typename T> struct IterativeSegTree {
    int n;
    vector<T> tree;

    // Default value for out-of-bounds or neutral element
    // For sum: 0, For max: -INF, For min: INF
    const T NEUTRAL = 0;

    IterativeSegTree(int size) {
        n = size;
        tree.assign(2 * n, NEUTRAL);
    }

    IterativeSegTree(const vector<T> &a) {
        n = a.size();
        tree.assign(2 * n, NEUTRAL);
        build(a);
    }

    // The combination operation. Change this for Max, Min, GCD, etc.
    T combine(T a, T b) { return a + b; }

    // O(N) build
    void build(const vector<T> &a) {
        for (int i = 0; i < n; i++) {
            tree[n + i] = a[i];
        }
        for (int i = n - 1; i > 0; i--) {
            tree[i] = combine(tree[i << 1], tree[i << 1 | 1]);
        }
    }

    // Point update: set a[p] = value in O(log N)
    void update(int p, T value) {
        for (tree[p += n] = value; p > 1; p >>= 1) {
            tree[p >> 1] = combine(tree[p], tree[p ^ 1]);
        }
    }

    // Range query in [l, r) in O(log N)

```



```

T query(int l, int r) {
    T resL = NEUTRAL, resR = NEUTRAL;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if (l & 1)
            resL = combine(resL, tree[l++]);
        if (r & 1)
            resR = combine(tree[--r], resR);
    }
    return combine(resL, resR);
}

};

int main() {
    // Example usage
    vector<long long> a = {5, 2, 8, 1, 3};
    IterativeSegTree<long long> seg(a);

    // Query sum from index 1 to 3 (values: 2, 8, 1) -> expects 11
    cout << "Sum [1, 4): " << seg.query(1, 4) << "\n";

    // Update index 2 to 10 (array becomes: 5, 2, 10, 1, 3)
    seg.update(2, 10);

    // Query sum from index 1 to 3 again (values: 2, 10, 1) -> expects 13
    cout << "Sum [1, 4): " << seg.query(1, 4) << "\n";

    return 0;
}

```

MO with DSU

DS/MO with DSU.cpp

```

struct Edge {
    int u, v;
};

struct DSU {
    vector<int> par, sz;
    int comp;
    struct State {
        int u, v; // u becomes child of v
        bool united;
    };
    vector<State> history;

    DSU(int n) {
        par.resize(n);
        iota(all(par), 0);
        sz.assign(n, 1);
        comp = n;
    }

    int find(int i) {
        while (par[i] != i)
            i = par[i];
        return i;
    }

    bool unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);
        if (root_i != root_j) {
            if (sz[root_i] < sz[root_j])
                swap(root_i, root_j);
            // Attach root_j to root_i
            history.push_back({root_j, root_i, true});
            par[root_j] = root_i;
            sz[root_i] += sz[root_j];
            comp--;
            return true;
        }
    }
}

```

```

    history.push_back({-1, -1, false}); // No merge happened
    return false;
}

void rollback() {
    if (history.empty())
        return;
    State last = history.back();
    history.pop_back();
    if (last.united) {
        // last.u was attached to last.v
        par[last.u] = last.u;
        sz[last.v] -= sz[last.u];
        comp++;
    }
}

int snapshot() { return history.size(); }

void rollbackTo(int snapshot) {
    while (history.size() > snapshot) {
        rollback();
    }
}

void reset(int n) {
    iota(par.begin(), par.end(), 0);
    sz.assign(n, 1);
    history.clear();
    comp = n;
}
};

int block_size;

struct Query {
    int l, r, idx;
    bool operator<(const Query &other) const {
        if (l / block_size != other.l / block_size)
            return l / block_size < other.l / block_size;
        return r < other.r;
    }
};

vector<int> mo(int n, const vector<Edge> &edges, vector<Query> queries) {
    int m = edges.size();
    block_size = sqrt(m) + 1;
    vector<int> answers(queries.size());
    sort(all(queries));

    DSU dsu(n);
    int num_blocks = (m + block_size - 1) / block_size;
    int q_idx = 0;

    for (int b = 0; b < num_blocks; b++) {
        dsu.reset(n);

        int block_end = min(m, (b + 1) * block_size);
        int cur_r = block_end;

        while (q_idx < queries.size() && queries[q_idx].l / block_size == b) {
            Query q = queries[q_idx];

            if (q.r < block_end) {
                int snap = dsu.snapshot();
                for (int i = q.l; i <= q.r; i++) {
                    dsu.unite(edges[i].u, edges[i].v);
                }
                answers[q.idx] = dsu.comp;
                dsu.rollbackTo(snap);
            }
        }
    }
}

```

```

        else {
            while (cur_r <= q.r) {
                dsu.unite(edges[cur_r].u, edges[cur_r].v);
                cur_r++;
            }
            int snap = dsu.snapshot();
            for (int i = q.l; i < block_end; i++) {
                dsu.unite(edges[i].u, edges[i].v);
            }

            answers[q.idx] = dsu.comp;
            dsu.rollbackTo(snap);
        }
        q_idx++;
    }
}
return answers;
}

void SM() {
    int n, m, q;
    cin >> n >> m >> q;

    vector<Edge> edges(m);
    for (int i = 0; i < m; i++) {
        cin >> edges[i].u >> edges[i].v;
        edges[i].u--;
        edges[i].v--;
    }

    vector<Query> queries(q);
    for (int i = 0; i < q; i++) {
        cin >> queries[i].l >> queries[i].r;
        queries[i].l--;
        queries[i].r--;
        queries[i].idx = i;
    }

    vector<int> ans = mo(n, edges, queries);

    for (auto x : ans)
        cout << x << endl;
}

signed main() {
    SMTYON
    int x_x = 1;
    cin >> x_x;
    while (x_x--) {
        SM();
    }

    return 0;
}

```

Mo

DS/Mo.cpp

```

// TIME: (n+q)*sqrt(n)
// TODO: data structure that have the answer
void add(int idx);    // TODO: add value at idx from data structure
void remove(int idx); // TODO: remove value at idx from data structure
int get_answer();     // TODO: extract the current answer of the data structure

int block_size; // TODO

struct Query {
    int l, r, idx;
    bool operator<(Query other) const {
        if (l / block_size != other.l / block_size)
            return l < other.l;
        return (l / block_size & 1) ? (r < other.r) : (r > other.r);
    }
};

vector<int> mo_s_algorithm(vector<Query> queries) {
    vector<int> answers(queries.size());
    sort(queries.begin(), queries.end());

    // TODO: initialize data structure

    int cur_l = 0;
    int cur_r = -1;
    // invariant: data structure will always reflect the range [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            add(cur_l);
        }
        while (cur_r < q.r) {
            cur_r++;
            add(cur_r);
        }
        while (cur_l < q.l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            remove(cur_r);
            cur_r--;
        }
        answers[q.idx] = get_answer();
    }
    return answers;
}

```

Persistant segment

DS/Persistent segment.cpp

```

struct Node {
    ll sum;
    int left_child;
    int right_child;
} NEUTRAL = {0, 0, 0};

struct PersistentSegTree {
    int size;
    vector<Node> tree;
    vector<int> roots;
    int node_count;

    PersistentSegTree(int n, int q) {
        size = 1;
        while (size < n)
            size *= 2;
    }
}

```

```

// Max nodes: ~ (2 * size) for base tree + (Q * log2(size)) for updates
int max_nodes = (2 * size) + (q * 25);
tree.resize(max_nodes);
roots.push_back(
    0); // 0-th index in roots can serve as a dummy or 0th version
node_count = 1;
}

// Creates a new node in the array pool and returns its index
int create_node(int sum, int lc, int rc) {
    int id = node_count++;
    tree[id].sum = sum;
    tree[id].left_child = lc;
    tree[id].right_child = rc;
    return id;
}

Node merge(Node a, Node b) {
    Node res;
    res.sum = a.sum + b.sum;
    // left and right children are handled during node creation, not merging
    return res;
}

// Returns the root of the built tree
int build(vector<int> &a, int lx, int rx) {
    if (rx - lx == 1) {
        if (lx < a.size())
            return create_node(a[lx], 0, 0);
        else
            return create_node(0, 0, 0);
    }

    int m = (lx + rx) / 2;
    int lc = build(a, lx, m);
    int rc = build(a, m, rx);

    Node merged = merge(tree[lc], tree[rc]);
    return create_node(merged.sum, lc, rc);
}

// Point update at index i. Returns the root of the NEW version.
int update(int i, int val, int x, int lx, int rx) {
    if (rx - lx == 1) {
        return create_node(val, 0, 0);
    }

    int m = (lx + rx) / 2;
    int lc = tree[x].left_child;
    int rc = tree[x].right_child;

    if (i < m) {
        lc = update(i, val, tree[x].left_child, lx, m);
    } else {
        rc = update(i, val, tree[x].right_child, m, rx);
    }

    Node merged = merge(tree[lc], tree[rc]);
    return create_node(merged.sum, lc, rc);
}

// Zero-based Range Query [l,r)
Node query(int l, int r, int x, int lx, int rx) {
    if (lx >= r || rx <= l || x == 0)
        return NEUTRAL;
    if (lx >= l && rx <= r)
        return tree[x];

    int m = (lx + rx) / 2;
    Node left_res = query(l, r, tree[x].left_child, lx, m);
    Node right_res = query(l, r, tree[x].right_child, m, rx);

```

```

    return merge(left_res, right_res);
}

void build(vector<int> &a) { roots.push_back(build(a, 0, size)); }
void update(int i, int val, int prev_version) {
    roots.push_back(update(i, val, roots[prev_version], 0, size));
}
ll query(int l, int r, int version) {
    return query(l, r, roots[version], 0, size).sum;
}
};

```

Segtree

DS/Segtree.cpp

```

struct Node {

} NEUTRAL;

struct SegTree {
    int size;
    vector<Node> tree;

    SegTree(int n) {
        size = 1;
        while (size < n)
            size *= 2;
        tree.resize(2 * size);
    }

    Node merage(const Node &a, const Node &b) {
        Node res;

        return res;
    }

    void build(vector<int> &a, int x, int lx, int rx) {
        if (rx - lx == 1) {
            if (lx < a.size()) {
            }
            return;
        }
        int m = (lx + rx) / 2;
        build(a, 2 * x + 1, lx, m);
        build(a, 2 * x + 2, m, rx);
        tree[x] = merage(tree[2 * x + 1], tree[2 * x + 2]);
    }

    // void update(int l, int r, int x, int lx, int rx) {
    //     if (lx >= r || rx <= l)
    //         return;
    //     if (lx >= l && rx <= r) {
    //         return;
    //     }
    //     int m = (lx + rx) / 2;
    //     update(l, r, 2 * x + 1, lx, m);
    //     update(l, r, 2 * x + 2, m, rx);
    //     tree[x] = merage(tree[2 * x + 1], tree[2 * x + 2]);
    // }

    // void update(int i, int v, int x, int lx, int rx) {
    //     if (rx - lx == 1) {

    //         return;
    //     }
    //     int m = (lx + rx) / 2;
    //     if (i < m)
    //         update(i, v, 2 * x + 1, lx, m);
    //     else
    //         update(i, v, 2 * x + 2, m, rx);
    //     tree[x] = merage(tree[2 * x + 1], tree[2 * x + 2]);
    // }

```

```

// }

// zero based Range Query [l,r)
// Node query(int l, int r, int x, int lx, int rx)
// {
//     if (lx >= r || rx <= l) return NEUTRAL;
//     if (lx >= l && rx <= r) return tree[x];
//     int m = (lx + rx)/2;
//     return merage(query(l, r, 2*x+1, lx, m), query(l, r, 2*x+2, m, rx));
// }

// zero based index Query i
// Node query(int i, int x, int lx, int rx)
// {
//     if(rx-lx == 1){
//         return tree[x];
//     }
//     int m = (lx + rx)/2;
//     if(i < m) return query(i, 2*x+1, lx, m);
//     else return query(i, 2*x+2, m, rx);
// }

void build(vector<int> &a) { build(a, 0, 0, size); }
// void update(int l, int r) { update(l, r, 0, 0, size); }
// void update(int i, int v) { update(i, v, 0, 0, size); }
// Node query(int l, int r) { return query(l, r, 0, 0, size); }
// Node query(int x) { return query(x, 0, 0, size); }
};

```

Segtree lazy

DS/Segtree_lazy.cpp

```

struct Node {
    bool lazy = 0;
} NEUTRAL;

struct SegTree {
    int size;
    vector<Node> tree;

    SegTree(int n) {
        size = 1;
        while (size < n)
            size *= 2;
        tree.resize(2 * size);
    }

    Node merage(Node a, Node b) {
        Node res;

        return res;
    }

    void build(vector<int> &a, int x, int lx, int rx) {
        if (rx - lx == 1) {
            if (lx < a.size()) {
            };
            return;
        }
        int m = (lx + rx) / 2;
        build(a, 2 * x + 1, lx, m);
        build(a, 2 * x + 2, m, rx);
        tree[x] = merage(tree[2 * x + 1], tree[2 * x + 2]);
    }

    void push(int x, int lx, int rx) {
        if (tree[x].lazy) {
            tree[x].lazy = 0;
        }
    }
};

```

```

void update(int l, int r, int x, int lx, int rx) {
    push(x, lx, rx);
    if (lx >= r || rx <= l)
        return;
    if (lx >= l && rx <= r) {
        tree[x].lazy = 1;
        push(x, lx, rx);
        return;
    }
    int m = (lx + rx) / 2;
    update(l, r, 2 * x + 1, lx, m);
    update(l, r, 2 * x + 2, m, rx);
    tree[x] = merge(tree[2 * x + 1], tree[2 * x + 2]);
}

// zero based Range Query [l,r)
// Node query(int l, int r, int x, int lx, int rx)
// {
//     push(x, lx, rx);
//     if (lx >= r || rx <= l) return NEUTRAL;
//     if (lx >= l && rx <= r) return tree[x];
//     int m = (lx + rx)/2;
//     return merge(query(l, r, 2*x+1, lx, m), query(l, r, 2*x+2, m, rx));
// }

// zero based index Query i
// Node query(int i, int x, int lx, int rx)
// {
//     push(x, lx, rx);
//     if(rx-lx == 1){
//         return tree[x];
//     }
//     int m = (lx + rx)/2;
//     if(i < m) return query(i, 2*x+1, lx, m);
//     else return query(i, 2*x+2, m, rx);
// }

void build(vector<int> &a) { build(a, 0, 0, size); }
void update(int l, int r) { update(l, r, 0, 0, size); }
Node query(int l, int r) { return query(l, r, 0, 0, size); }
};

```


Sparse table

DS/Sparse table.cpp

```
struct Sparse {
    vector<int> log;
    vector<vector<int>> table;
    Sparse(int n, vector<int> &a) {
        log.resize(n + 5);
        log[1] = 0;
        // -> i -> 10101011
        // -> i / 2 -> 01010101
        for (int i = 2; i <= n; i++) {
            log[i] = log[i / 2] + 1;
        }

        table = vector<vector<int>>(n + 5, vector<int>(log[n] + 5, 0));
        // table[N][log(N)]
        for (int i = 0; i < n; i++)
            table[i][0] = a[i];
        // levels
        for (int j = 1; j <= log[n]; j++) {

            for (int k = 0; k + (1 << j) <= n; k++) {
                table[k][j] = __gcd(table[k][j - 1], table[(1 << (j - 1)) + k][j - 1]);
            }
        }
    }
    int query(int l, int r) {
        // l--, r--; // For 1 based query
        int x = r - l + 1;
        x = log[x];
        return __gcd(table[l][x], table[r - (1 << x) + 1][x]);
    }
};
```

Sqrt Decomposition

DS/Sqrt Decomposition.cpp

```
int n;
vector<int> a(n);

// preprocessing
int len = (int)sqrt(n + .0) + 1; // size of the block and the number of blocks
vector<int> b(len);
for (int i = 0; i < n; ++i)
    b[i / len] += a[i];

// answering the queries
// option 1:
for (;;) {
    int l, r;
    // read input data for the next query
    int sum = 0;
    for (int i = l; i <= r; i) {
        if (i % len == 0 && i + len - 1 <= r) {
            // if the whole block starting at i belongs to [l, r]
            sum += b[i / len];
            i += len;
        } else {
            sum += a[i];
            ++i;
        }
    }
}

// option 2:
int sum = 0;
int c_l = l / len, c_r = r / len;
if (c_l == c_r)
    for (int i = l; i <= r; ++i)
        sum += a[i];
else {
    for (int i = l, end = (c_l + 1) * len - 1; i <= end; ++i)
        sum += a[i];
    for (int i = c_l + 1; i <= c_r - 1; ++i)
        sum += b[i];
    for (int i = c_r * len; i <= r; ++i)
        sum += a[i];
}
```

String Trie

DS/String_Trie.cpp

```
struct Trie {
    struct Node {
        int p = 0;
        int ch[26]{};
    };
    vector<Node> tree;
    Trie() { tree.emplace_back(); }

    void insert(string s) {
        int node = 0;
        for (auto &it : s) {
            int c = it - 'a';
            if (tree[node].ch[c] == 0) {
                tree[node].ch[c] = tree.size();
                tree.emplace_back();
            }
            node = tree[node].ch[c];
            tree[node].p++;
        }
    }

    void erase(string s) {
        int node = 0;
        for (auto &it : s) {
            int c = it - 'a';
            node = tree[node].ch[c];
            tree[node].p--;
        }
    }

    int query(string s) {
        int node = 0;
        int ret = 0;
        for (auto &it : s) {
            int c = it - 'a';
            if (tree[tree[node].ch[c]].p == 0)
                return ret;
            node = tree[node].ch[c];
            ret++;
        }
        return ret;
    }
};
```

Graph (7)

Graph

Bridges & Articulation Points

Graph/Bridges & Articulation Points.cpp

```
#include <bits/stdc++.h>
using namespace std;

// =====
// Bridges and Articulation Points (Cut-Vertices)
//
// O(V + E)
// Graph Type: Undirected Graphs only.
//
// Usage:
// 1. Set 'n' to the number of nodes.
// 2. Call UndirectedGraph::init(n).
// 3. Add UNDIRECTED edges using UndirectedGraph::add_edge(u, v). (0-indexed)
// 4. Call UndirectedGraph::find_bridges_and_points().
//
// Output:
// - UndirectedGraph::bridges: A vector of pairs {u, v} representing cut-edges.
// - UndirectedGraph::is_art_point[u]: A boolean array. True if node 'u' is a cut-vertex.
//
// Note: This template handles disconnected graphs automatically.
// =====

namespace UndirectedGraph {
    int n, timer;
    vector<vector<int>>> adj;
    vector<int> tin, low;

    // Outputs
    vector<pair<int, int>> bridges;
    vector<bool> is_art_point;

    void init(int nodes) {
        n = nodes; timer = 0;
        adj.assign(n, vector<int>());
        tin.assign(n, -1);
        low.assign(n, -1);
        bridges.clear();
        is_art_point.assign(n, false);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int u, int p = -1) {
        tin[u] = low[u] = timer++;
        int children = 0;

        for (int v : adj[u]) {
            if (v == p) continue; // Don't go immediately back to parent

            if (tin[v] != -1) {
                // Back-edge found
                low[u] = min(low[u], tin[v]);
            } else {
                // Forward-edge (tree edge)
                dfs(v, u);
                low[u] = min(low[u], low[v]);

                // --- BRIDGE CONDITION ---
                // If the lowest reachable node from v is strictly below u,
                // the edge u-v is the only way out for v.
                if (low[v] > tin[u]) {
```

```
        bridges.push_back({u, v});
    }

    // --- ARTICULATION POINT CONDITION ---
    // If we are not the root, and v cannot reach above u, u is a bottleneck.
    if (low[v] >= tin[u] && p != -1) {
        is_art_point[u] = true;
    }
    children++;
}

// --- ROOT ARTICULATION POINT CONDITION ---
// The root of the DFS tree is an articulation point ONLY if it has >1 independent children.
if (p == -1 && children > 1) {
    is_art_point[u] = true;
}

}

void find_bridges_and_points() {
    for (int i = 0; i < n; i++) {
        if (tin[i] == -1) {
            dfs(i); // Process each connected component
        }
    }
}

}
```

Floyd Warshall

Graph/Floyd_Warshall.cpp

```
void floydWarshall(vector<vector<int>> &dist) {
    int V = dist.size();

    // Add all vertices one by one to
    // the set of intermediate vertices.
    for (int k = 0; k < V; k++) {

        // Pick all vertices as source one by one
        for (int i = 0; i < V; i++) {

            // Pick all vertices as destination
            // for the above picked source
            for (int j = 0; j < V; j++) {

                // shortest path from
                // i to j
                if (dist[i][k] != 1e8 && dist[k][j] != 1e8)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}

int main() {
    int INF = 100000000;
    vector<vector<int>> dist = {{0, 4, INF, 5, INF},
                               {INF, 0, 1, INF, 6},
                               {2, INF, 0, 3, INF},
                               {INF, INF, 1, 0, 2},
                               {1, INF, INF, 4, 0}};

    floydWarshall(dist);
    for (int i = 0; i < dist.size(); i++) {
        for (int j = 0; j < dist.size(); j++) {
            cout << dist[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

LCA

Graph/LCA.cpp

```

struct LCA {
    const int LG = 20;
    int timer = 0;
    vector<int> in, out;
    vector<vector<int>> par;
    LCA(int n, int root, vector<vector<int>> &adj) {
        in = out = vector<int>(n + 2);
        par = vector<vector<int>>(n + 2, vector<int>(LG));
        dfs(root, root, adj);
    }
    void dfs(int node, int p, vector<vector<int>> &adj) {

        in[node] = timer++;
        par[node][0] = p;
        for (int j = 1; j < LG; j++) {
            par[node][j] = par[par[node][j - 1]][j - 1];
        }
        for (auto &child : adj[node]) {
            if (child == p)
                continue;
            dfs(child, node, adj);
        }
        out[node] = timer++;
    }

    int kth(int u, int k) {
        for (int i = LG - 1; ~i; i--) {
            if ((k >> i) & 1) {
                u = par[u][i];
            }
        }
        return u;
    }

    bool isPar(int node, int p) {
        return in[p] <= in[node] && out[p] >= out[node];
    }

    int lca(int u, int v) {
        if (isPar(v, u))
            return u;
        for (int i = LG - 1; ~i; i--) {
            if (!isPar(v, par[u][i])) {
                u = par[u][i];
            }
        }
        return par[u][0];
    }
};

```

SCC

Graph/SCC.cpp

```

#include <bits/stdc++.h>
using namespace std;

// =====
// Tarjan's SCC
//
// O(V + E)
// Graph Type: Directed Graphs only.
//
// Usage:
// 1. Set 'n' to the number of nodes.
// 2. Call SCC::init(n).
// 3. Add directed edges to SCC::adj using SCC::adj[u].push_back(v). (0-indexed)
// 4. Call SCC::build().
//

```

```

// Output:
// - SCC::scc_cnt: Total number of components.
// - SCC::scc_id[u]: The ID of the component node 'u' belongs to (0 to scc_cnt-1).
// - SCC::comp: comp[i] contains all original nodes in component ID 'i'.
// - SCC::condensed_adj: The new DAG where components are contracted into super-nodes.
// =====

namespace SCC {
    int n, timer, scc_cnt;
    vector<vector<int>> adj, comp, condensed_adj;
    vector<int> scc_id, tin, low;
    vector<bool> on_stack;
    stack<int> st;

    void init(int nodes) {
        n = nodes; timer = 0; scc_cnt = 0;
        adj.assign(n, vector<int>());
        comp.clear(); condensed_adj.clear();
        scc_id.assign(n, -1);
        tin.assign(n, -1); low.assign(n, -1);
        on_stack.assign(n, false);
        while (!st.empty()) st.pop();
    }

    void dfs(int u) {
        tin[u] = low[u] = timer++;
        st.push(u);
        on_stack[u] = true;

        for (int v : adj[u]) {
            if (tin[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (on_stack[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }

        if (low[u] == tin[u]) {
            comp.push_back({});
            while (true) {
                int v = st.top(); st.pop();
                on_stack[v] = false;
                scc_id[v] = scc_cnt;
                comp.back().push_back(v);
                if (u == v) break;
            }
            scc_cnt++;
        }
    }

    void build() {
        for (int i = 0; i < n; i++) {
            if (tin[i] == -1) dfs(i);
        }

        // Build the condensed DAG
        condensed_adj.assign(scc_cnt, vector<int>());
        for (int u = 0; u < n; u++) {
            for (int v : adj[u]) {
                if (scc_id[u] != scc_id[v]) {
                    condensed_adj[scc_id[u]].push_back(scc_id[v]);
                }
            }
        }

        // Remove duplicate edges in the condensed graph
        for (int i = 0; i < scc_cnt; i++) {
            sort(condensed_adj[i].begin(), condensed_adj[i].end());
            condensed_adj[i].erase(unique(condensed_adj[i].begin(), condensed_adj[i].end()), condensed_adj[i].end());
        }
    }
}

```



```

    }
}

```

Sack

Graph/Sack.cpp

```

/*
The Problem: You are given a tree with N nodes.
Every node has a specific color. For every single node in the tree,
you need to output the number of distinct colors present in its subtree.
*/
#include <iostream>
#include <vector>
using namespace std;

const int MAXN = 1e5+5;
vector<int> adj[MAXN];
int color[MAXN], ans[MAXN];

// Tree properties
int sz[MAXN]; // Size of the subtree
int heavy[MAXN]; // The heavy child of a node (-1 if leaf)

// Global state tracking
int cnt[MAXN]; // Global frequency array
int distinct_colors = 0; // Running answer for the global state

// -----
// PHASE 1: Precalculate subtree sizes and heavy children
// -----
void dfs_sz(int u, int p) {
    sz[u] = 1;
    heavy[u] = -1;
    int max_sub = 0;

    for (int v : adj[u]) {
        if (v != p) {
            dfs_sz(v, u);
            sz[u] += sz[v];

            // If this child is the biggest so far, mark it as heavy
            if (sz[v] > max_sub) {
                max_sub = sz[v];
                heavy[u] = v;
            }
        }
    }
}

// Helper function to add/remove a subtree from the global state
// 'val' is 1 to add, -1 to remove.
// 'skip' allows us to completely ignore the heavy child's subtree when combining.
void update_subtree(int u, int p, int val, int skip) {
    // Process current node
    if (val == 1 && cnt[color[u]] == 0) distinct_colors++;
    if (val == -1 && cnt[color[u]] == 1) distinct_colors--;
    cnt[color[u]] += val;

    // Process all children EXCEPT the 'skip' node
    for (int v : adj[u]) {
        if (v != p && v != skip) {
            update_subtree(v, u, val, skip);
        }
    }
}

// -----
// PHASE 2: The actual Sack (DSU on Tree) logic
// -----
void dfs_sack(int u, int p, bool keep) {
    // Step 1: Process all LIGHT children first.

```

```

// Pass 'keep = false' so their data is wiped from the global state.
for (int v : adj[u]) {
    if (v != p && v != heavy[u]) {
        dfs_sack(v, u, false);
    }
}

// Step 2: Process the HEAVY child.
// Pass 'keep = true' so its massive subtree data stays in the global array.
if (heavy[u] != -1) {
    dfs_sack(heavy[u], u, true);
}

// Step 3: Combine.
// Add the current node 'u' and all its light subtrees into the global state.
// We pass heavy[u] as the 'skip' parameter because it's ALREADY in the array.
update_subtree(u, p, 1, heavy[u]);

// Step 4: The global state now perfectly represents the subtree of 'u'. Record answer.
ans[u] = distinct_colors;

// Step 5: Clean up.
// If 'u' is a light child itself (keep == false), wipe it from the global state
// before we return to 'u's parent.
if (!keep) {
    update_subtree(u, p, -1, -1); // pass -1 to skip nothing
}
}

```

matching

Graph/matching.cpp

```

#include <bits/stdc++.h>
using namespace std;

struct BipartiteMatcher {
    // Time Complexity:  $O(V * E)$  where  $V$  = left nodes,  $E$  = edges
    // König's Theorem: Minimum Vertex Cover (MVC) = Maximum Bipartite Matching (MBM)
    // Complement: Maximum Independent Set (MIS) = (Total Nodes) - MBM

    int n, m;
    vector<vector<int>> adj;
    vector<int> cap, matchL;
    vector<vector<int>> matchR; // matchR[v] = left nodes matched to right node v
    vector<bool> vis;

    // Default capacity is 1 (standard 1-to-1 bipartite matching)
    BipartiteMatcher(int n, int m) : n(n), m(m), adj(n), cap(m, 1), matchL(n, -1), matchR(m) {}

    void addEdge(int u, int v) { adj[u].push_back(v); }

    void setCapacity(int v, int c) { cap[v] = c; }

    bool dfs(int u) {
        for (int v : adj[u]) {
            if (vis[v]) continue;
            vis[v] = true;

            // Base case: Right node has free capacity
            if (matchR[v].size() < cap[v]) {
                matchR[v].push_back(u);
                matchL[u] = v;
                return true;
            }

            // Recursive case: Try to reassign an existing match
            for (int i = 0; i < matchR[v].size(); ++i) {
                int prev_u = matchR[v][i];

```

```

        if (dfs(prev_u)) {
            matchR[v][i] = u;
            matchL[u] = v; // DFS updates prev_u's matchL automatically
            return true;
        }
    }
}
return false;
}

int maxMatching() {
    int match = 0;
    for (int u = 0; u < n; u++) {
        vis.assign(m, false);
        if (dfs(u)) match++;
    }
    return match;
}

// Returns {coverL, coverR}
pair<vector<int>, vector<int>> minVertexCover() {
    vector<bool> visL(n, false), visR(m, false);
    queue<int> q;

    // Start BFS from all unmatched left nodes
    for (int u = 0; u < n; u++) {
        if (matchL[u] == -1) {
            visL[u] = true;
            q.push(u);
        }
    }

    while (q.size()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!visR[v]) {
                visR[v] = true;
                for (int matched_u : matchR[v]) {
                    if (!visL[matched_u]) {
                        visL[matched_u] = true;
                        q.push(matched_u);
                    }
                }
            }
        }
    }

    vector<int> coverL, coverR;
    for (int u = 0; u < n; u++) if (!visL[u]) coverL.push_back(u);
    for (int v = 0; v < m; v++) if (visR[v]) coverR.push_back(v);
    return {coverL, coverR};
}

// Returns {indL, indR}
pair<vector<int>, vector<int>> maxIndependentSet() {
    auto [coverL, coverR] = minVertexCover();

    vector<bool> inCovL(n, false), inCovR(m, false);
    for (int u : coverL) inCovL[u] = true;
    for (int v : coverR) inCovR[v] = true;

    vector<int> indL, indR;
    for (int u = 0; u < n; u++) if (!inCovL[u]) indL.push_back(u);
    for (int v = 0; v < m; v++) if (!inCovR[v]) indR.push_back(v);
    return {indL, indR};
}

bool isPerfectMatching() {
    for (int u = 0; u < n; u++) {
        if (matchL[u] == -1) return false;
    }
    return true;
}

```

```

    }
};

```

small to large

Graph/small_to_large.cpp

```

#include <iostream>
#include <vector>
#include <map>

using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN];
int color[MAXN];
int ans[MAXN];

// Array of maps to hold the data for each node
map<int, int> mp[MAXN];

void dfs_small_to_large(int u, int p) {
    // 1. Add the current node's color to its own map
    mp[u][color[u]]++;

    for (int v : adj[u]) {
        if (v == p) continue;

        // 2. Process the child
        dfs_small_to_large(v, u);

        // 3. SMALL-TO-LARGE TRICK:
        // If the child's map is larger, swap them.
        // std::swap works in O(1) time by just swapping internal pointers!
        if (mp[v].size() > mp[u].size()) {
            swap(mp[u], mp[v]);
        }

        // 4. Merge the smaller map (v) into the larger map (u)
        for (auto const& [col, count] : mp[v]) {
            mp[u][col] += count;
        }

        // 5. Free up memory to avoid Memory Limit Exceeded
        // Since we swapped earlier, mp[v] might contain the larger data,
        // so we must clear it now that its elements are safely in mp[u].
        mp[v].clear();
    }

    // 6. The number of distinct colors in node u's subtree is the size of the map
    ans[u] = mp[u].size();
}

```

Math (13)

math

Counting

math/Counting.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(), a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);
const ll mod = 1e9 + 7, inf = 1e9, MAXN = 1e5 + 5;

// Basic Modular Operations:
ll add(ll a, ll b) { return (a + b + mod) % mod; }
ll mul(ll a, ll b) { return (a * b) % mod; }
ll fp(ll base, ll power) {
    if (!power)
        return 1;
    ll res = fp(base, power / 2);
    res = mul(res, res);
    if (power & 1)
        res = mul(res, base);
    return res;
}
ll inverse(ll n) { return fp(n, mod - 2); }
ll divide(ll a, ll b) { return mul(a, inverse(b)); }

// Combinatorics (nCr, nPr, Stars & Bars):
ll fact[MAXN], invFact[MAXN];
void precompute_factorials() {
    fact[0] = 1;
    invFact[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        fact[i] = mul(fact[i - 1], i);
    }

    invFact[MAXN - 1] = inverse(fact[MAXN - 1]);
    for (int i = MAXN - 2; i >= 1; i--) {
        invFact[i] = mul(invFact[i + 1], i + 1);
    }
}

ll nCr(int n, int r) {
    if (r < 0 || r > n)
        return 0;
    return mul(fact[n], mul(invFact[r], invFact[n - r]));
}

ll nPr(int n, int r) {
    if (r < 0 || r > n)
        return 0;
    return mul(fact[n], invFact[n - r]);
}

// Stars and Bars
// Problem: Distribute n identical items into k distinct bins.
ll stars_andBars(int n, int k) {
    if (k == 0)
        return (n == 0);
    return nCr(n + k - 1, k - 1);
}

ll catalan(int n) {
    ll numer = nCr(2 * n, n);

```

```

    ll denom = inverse(n + 1);
    return mul(number, denom);
}

// Works for n up to 10^18, provided r is small (< 10^6)
// Complexity: O(r)
ll nCr_large_n(ll n, int r) {
    if (r < 0 || r > n)
        return 0;
    if (r == 0 || r == n)
        return 1;
    if (r > n / 2)
        r = n - r;

    ll number = 1;
    for (int i = 0; i < r; i++) {
        number = mul(number, (n - i) % mod);
    }

    return mul(number, invFact[r]);
}

// Returns number of ways to arrange n items such that none are in their
// original spot. Complexity: O(N) precompute, O(1) query.
vector<ll> Dn;
void precompute_derangements(int n) {
    Dn.resize(n + 1);
    Dn[0] = 1;
    Dn[1] = 0;
    for (int i = 2; i <= n; i++) {
        Dn[i] = mul(i - 1, add(Dn[i - 1], Dn[i - 2]));
    }
}

signed main() {
    SMTYON
    int x_x = 1;
    // cin >> x_x;
    while (x_x--) {

    }

    return 0;
}

```

Expected Value

math/Expected Value.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define ld long double
/*
E(x) = x * P(x).
E(x+y) = E(x) + E(y) -> dependant or independant.
E(x*y) = E(x) * E(y) -> only if independant.
E(x/y) != E(x) / E(y).
*/

ld fp(ld a, ll p){
    if(p == 0) return 1.0;
    ld res = fp(a,p/2);
    res *= res;
    if(p&1) res *= a;
    return res;
}

```

BigInt

math/FFT/NTT/FWHT/BigInt.cpp

```
string mul_two_big_int(const string &s1, const string &s2) {
    int n = s1.size(), m = s2.size();

    vector<int> poly1(n), poly2(m);
    for (int i = 0; i < n; ++i) {
        poly1[n-i-1] = s1[i] - '0';
    }

    for (int i = 0; i < m; ++i) {
        poly2[m-i-1] = s2[i] - '0';
    }

    vector<int> ans = multiply(poly1, poly2);
    int k = ans.size();

    for (int i = 0; i < k - 1; ++i) {
        ans[i + 1] += ans[i] / 10;
        ans[i] = ans[i] % 10;
    }

    string final = to_string(ans[k - 1]);
    for (int i = k - 2; i >= 0; --i) {
        final += (char)(ans[i] + '0');
    }

    for (int i = 0; i < k; ++i) {
        if(final[i] != '0') return final.substr(i);
    }
    return "0";
}

string power_of_big_int(string s, int p) {
    string ans = "1";
    while (p) {
        if(p&1) ans = mul_two_big_int(ans, s);
        s = mul_two_big_int(s, s);
        p >>= 1;
    }
    return ans;
}
```

FFT

math/FFT/NTT/FWHT/FFT.cpp

```

#define int long long
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

```

FFT mod

math/FFT/NTT/FWHT/FFT_mod.cpp

```

#include <iostream>
#include <vector>
#include <complex>
#include <cmath>
using namespace std;

using ll = long long;
using C = complex<double>;

```



```

void fft(vector<C>& a) {
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);

    for (static int k = 2; k < n; k *= 2) {
        R.resize(n);
        rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        for (int i = k; i < 2 * k; i++) {
            rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
        }
    }

    vector<int> rev(n);
    for (int i = 0; i < n; i++) {
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    }
    for (int i = 0; i < n; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }

    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                // Hand-rolled complex multiplication for speed
                auto x = (double *)&rt[j + k];
                auto y = (double *)&a[i + j + k];
                C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);

                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
        }
    }
}

/*
 * MAIN FUNCTION: convMod<MODULO>(a, b)
 * Computes the convolution of two arrays modulo M using the Split FFT trick.
 * Resistant to floating point precision errors up to M <= 10^9.
 */
template<int M>
vector<int> convMod(const vector<int> &a, const vector<int> &b) {
    if (a.empty() || b.empty()) return {};

    vector<int> res((int)a.size() + (int)b.size() - 1);
    int B = 32 - __builtin_clz((int)res.size());
    int n = 1 << B;
    int cut = int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);

    for (int i = 0; i < (int)a.size(); i++) {
        L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    }
    for (int i = 0; i < (int)b.size(); i++) {
        R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    }

    fft(L);
    fft(R);

    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / C(0, 1);
    }

    fft(outl);
    fft(outs);

    for (int i = 0; i < (int)res.size(); i++) {

```

```

    ll av = ll(real(outl[i]) + 0.5);
    ll cv = ll(imag(outs[i]) + 0.5);
    ll bv = ll(imag(outl[i]) + 0.5) + ll(real(outs[i]) + 0.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
}

return res;
}

int main() {
    vector<int> a = {1, 2, 3};
    vector<int> b = {4, 5};

    // Calculates the convolution modulo 10^9+7
    vector<int> result = convMod<1000000007>(a, b);

    // result[k] = sum(a[i] * b[j]) % 1000000007 for i+j == k

    return 0;
}

```

FWHT

math/FFT/NTT/FWHT/FWHT.cpp

```

ll mod = 998244353; // Set your modulo here
int add(int a, int b) { return (a + b) % mod;}
int sub(int a, int b) { return (a - b + mod) % mod;}
ll poww(ll a, ll b) {
    ll ret = 1;
    while (b) {
        if (b & 1) ret = ret * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ret;
}

void fwht(vector<int> &a, bool inv, int f) {
    int sz = a.size();
    for (int len = 1; 2 * len <= sz; len <= 1) {
        for (int i = 0; i < sz; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                int x = a[i + j];
                int y = a[i + j + len];

                if (f == 0) { // Bitwise AND
                    if (!inv) {
                        a[i + j] = add(x, y);
                        a[i + j + len] = y;
                    } else {
                        a[i + j] = sub(x, y);
                        a[i + j + len] = y;
                    }
                }
            }
        }
        else if (f == 1) { // Bitwise OR
            if (!inv) {
                a[i + j] = x;
                a[i + j + len] = add(x, y);
            } else {
                a[i + j] = x;
                a[i + j + len] = sub(y, x);
            }
        }
        else if (f == 2) { // Bitwise XOR
            a[i + j] = add(x, y);
            a[i + j + len] = sub(x, y);
        }
    }
}
}
}
}

```

```

/*
 * MAIN BLACK BOX FUNCTION: mul(a, b, f)
 *
 * WHAT IT DOES:
 * Calculates the number of ways to combine elements from array `a`
 * and array `b` using a bitwise operation `f`.
 *
 * FLAG OPTIONS (f):
 * - 0 : AND (i & j == k)
 * - 1 : OR  (i | j == k)
 * - 2 : XOR (i ^ j == k)
 *
 * HOW TO FORMAT INPUTS:
 * - a[i] = the number of items you have with value `i`.
 * - b[j] = the number of items you have with value `j`.
 * NOTE: The array size represents the maximum possible value.
 * If your highest number is 1000, your array must be size 1001.
 *
 * WHAT THE OUTPUT MEANS:
 * Returns a vector `result` where:
 * - result[k] = total combinations of one item from `a` and one
 *               item from `b` where (i OP j) = k.
 */
vector<int> mul(vector<int> a, vector<int> b, int f) {
    // 1. Auto-pad arrays to the nearest power of 2
    int max_sz = max(a.size(), b.size());
    int sz = 1;
    while (sz < max_sz) sz <= 1;
    a.resize(sz, 0);
    b.resize(sz, 0);

    // 2. Transform both arrays to point-value representation
    fwht(a, false, f);
    fwht(b, false, f);

    // 3. Multiply them in O(N)
    vector<int> c(sz);
    for (int i = 0; i < sz; ++i) {
        c[i] = 1ll * a[i] * b[i] % mod;
    }

    // 4. Transform back to coefficient representation
    fwht(c, true, f);

    // 5. Normalize (ONLY XOR requires division by N)
    if (f == 2) {
        int sz_inv = poww(sz, mod - 2);
        for (int i = 0; i < sz; ++i) {
            c[i] = 1ll * c[i] * sz_inv % mod;
        }
    }
    return c;
}

```

NTT

math/FFT/NTT/FWHT/NTT.cpp

```

// USE generator TO CLACULATE THE NEW root FOR YOUR mod !!!!!!!!!!!

//mod has to be a prime of the form c * 2^k + 1 (like 998244353)
// Default is set to 998244353 and its primitive root 3.
#define int ll
ll mod = 998244353;
ll root = 3;

int modpow(int b, int e, int m = mod) {
    int ans = 1;
    for (; e; b = (ll)b * b % m, e /= 2)
        if (e & 1) ans = (ll)ans * b % m;
    return ans;
}

```

```

}

// Primitive Root of the mod of form 2^a * b + 1
int generator () {
    vector<int> fact;
    int phi = mod-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=mod; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= modpow (res, phi / fact[i]) != 1;
        if (ok) return res;
    }
    return -1;
}

void ntt(vector<int> &a) {
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    vector<int> rt(2, 1);
    for (int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        int z[] = {1, modpow(root, mod >> s, mod)};
        for (int i = k; i < 2*k; ++i) rt[i] = (ll)rt[i / 2] * z[i & 1] % mod;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; ++j) {
                int z = (ll)rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
        }
    }
}

/*
 * MAIN FUNCTION FOR NTT-FRIENDLY PRIMES (e.g., 998244353)
 *
 * Usage: vector<int> res = conv(a, b);
 * Returns array where res[k] = sum(a[i] * b[j]) % mod for i+j == k.
 * Automatically sizes the output array to exactly a.size() + b.size() - 1.
 */
vector<int> conv(const vector<int> &a, const vector<int> &b) {
    if (a.empty() || b.empty()) return {};
    int s = (int)a.size() + (int)b.size() - 1, B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = modpow(n, mod - 2, mod);
    vector<int> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    for (int i = 0; i < n; ++i) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

// =====
// PART 2: ARBITRARY MODULO (Use if modulo is 10^9+7 or not NTT-friendly)
// =====

```

```

// The modulo you actually want your final answer in
int desired_mod = 1000000007;

// Three standard NTT-friendly primes and their roots
const int mod1 = 167772161, root1 = 3;
const int mod2 = 469762049, root2 = 3;
const int mod3 = 754974721, root3 = 11;

/* Chinese Remainder Theorem to stitch the 3 convolutions together */
int CRT(int a, int b, int c, int m1, int m2, int m3) {
    __int128 M = (__int128)m1*m2*m3;
    ll M1 = (ll)m2*m3;
    ll M2 = (ll)m1*m3;
    ll M3 = (ll)m2*m1;

    int M_1 = modpow(M1%m1, m1 - 2, m1);
    int M_2 = modpow(M2%m2, m2 - 2, m2);
    int M_3 = modpow(M3%m3, m3 - 2, m3);

    __int128 ans = (__int128)a*M1*M_1;
    ans += (__int128)b*M2*M_2;
    ans += (__int128)c*M3*M_3;

    return (ans % M) % desired_mod;
}

/*
 * MAIN FUNCTION FOR NON-FRIENDLY PRIMES (e.g., 10^9+7)
 *
 * Usage: vector<int> res = arbitrary_mod_conv(a, b);
 * Runs the convolution 3 times and stitches the result.
 * Expected to be ~3x slower than conv().
 */
vector<int> arbitrary_mod_conv(const vector<int>& a, const vector<int>& b) {
    // 1st pass
    mod = mod1; root = root1;
    vector<int> res1 = conv(a, b);

    // 2nd pass
    mod = mod2; root = root2;
    vector<int> res2 = conv(a, b);

    // 3rd pass
    mod = mod3; root = root3;
    vector<int> res3 = conv(a, b);

    // Restore global mod just in case it's used elsewhere
    mod = 998244353; root = 3;

    int n = max({res1.size(), res2.size(), res3.size()});
    vector<int> final_res(n);

    for (int i = 0; i < n; i++) {
        int v1 = (i < res1.size()) ? res1[i] : 0;
        int v2 = (i < res2.size()) ? res2[i] : 0;
        int v3 = (i < res3.size()) ? res3[i] : 0;

        final_res[i] = CRT(v1, v2, v3, mod1, mod2, mod3);
    }

    return final_res;
}

```

math/FFT/NTT/FWHT/poly_pow.cpp

poly pow

```

vector<int> poly_pow(vector<int> poly, int p, int limit = 1e9) {
    vector<int> ans{1};
    while (p) {
        if(p&1) ans = conv(ans, poly);
        poly = conv(poly, poly);
        ans.resize(limit + 1);
        poly.resize(limit + 1);
        p >>= 1;
    }
    return ans;
}

```

math/FFT/NTT/FWHT/string_matching.cpp

string matching

```

// FFT
// find all occurrences of a pattern inside a text (string s), even if the pattern contains
// wildcards (like *) that can match any character.
void solve() {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<int> poly1(n), poly2(m);

    // ans_match[i] represents the number of successfully matched characters
    // between the pattern patt and a substring of s that starts at index i
    vector<int> ans_match(n);

    for (int i = 0; i < 26; ++i) {
        for (int j = 0; j < n; ++j) {
            poly1[j] = (s[j] - 'a') == i;
        }
        for (int j = 0; j < m; ++j) {
            poly2[j] = (patt[m-j-1] - 'a') == i;
        }
        vector<int> ans = multiply(poly1, poly2);
        for (int j = 0; j < n; ++j) {
            ans_match[j] += ans[m-1+j];
        }
    }

    int tot = 0;
    vector<int> pos;
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');
    for (int i = 0; i < n; ++i) {
        if(ans_match[i] == m - wild_cnt) {
            ++tot;
            pos.push_back(i);
        }
    }

    for(auto & p : pos) cout << p << " ";
    cout << "\n";

}

using cd = complex<double>;
const double PI = acos(-1), eps = 5e-4; // If you get a wrong answer you can change the eps lower of higher till you pass

// faster than the function above but it uses double :(
vector<cd> multiply(vector<cd> const& a, vector<cd> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <<= 1;
    fa.resize(n);

```

```

fb.resize(n);

fft(fa, false);
fft(fb, false);
for (int i = 0; i < n; i++)
    fa[i] *= fb[i];
fft(fa, true);

return fa;
}

void solve() {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<cd> poly1(n), poly2(m);

    for (int i = 0; i < n; ++i) {
        double angle = 2*PI*(s[i]-'a')/26;
        poly1[i] = cd(cos(angle), sin(angle));
    }
    for (int i = 0; i < m; ++i) {
        if(patt[m-i-1] == '*') poly2[i] = cd(0,0); // Wild Card
        else {
            double angle = 2*PI*(patt[m-i-1]-'a')/26;
            poly2[i] = cd(cos(angle), -sin(angle));
        }
    }

    vector<cd> ans = multiply(poly1, poly2);
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');

    int tot = 0;
    vector<int> pos;
    for (int i = 0; i < n; ++i) {
        if(fabs(ans[m-1+i].real() - (m - wild_cnt)) < eps && fabs(ans[m-1+i].imag()) < eps) {
            ++tot;
            pos.push_back(i);
        }
    }

    for(auto & p : pos) cout << p << " ";
    cout << "\n";

}

```

Laws

math/Laws.cpp

```

/*
(a+b)^n = r from_0_to_n -> summation ( nCr * a^(n-r) * b^r ).
*/

```

Math

math/Math.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(),a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7, inf = 1e9, MAXN = 1e5+5;

ll add(ll a, ll b) {return (a + b) % mod;}
ll mul(ll a, ll b) {return (a * b) % mod;}

```

```

// Solves  $a^x = b \pmod{m}$ 
// Returns -1 if no solution found.
// Complexity:  $O(\sqrt{m} \cdot \log(m))$ 
ll discrete_log(ll a, ll b, ll m) {
    a %= m; b %= m;
    ll k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k) return add;
        if (b % g) return -1;
        b /= g; m /= g; add++;
        k = (k * 111 * a / g) % m;
    }

    ll n = sqrt(m) + 1;
    ll an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    map<ll, int> vals;
    ll cur = b;
    for (int q = 0; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 111 * a) % m;
    }

    cur = k;
    for (int p = 1; p <= n; ++p) {
        cur = (cur * 111 * an) % m;
        if (vals.count(cur)) {
            ll ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}

// Fibonacci (Matrix Exponentiation)
// -----
void multiply_matrix(ll F[2][2], ll M[2][2]) {
    ll a = add(mul(F[0][0], M[0][0]), mul(F[0][1], M[1][0]));
    ll b = add(mul(F[0][0], M[0][1]), mul(F[0][1], M[1][1]));
    ll c = add(mul(F[1][0], M[0][0]), mul(F[1][1], M[1][0]));
    ll d = add(mul(F[1][0], M[0][1]), mul(F[1][1], M[1][1]));

    F[0][0] = a; F[0][1] = b;
    F[1][0] = c; F[1][1] = d;
}

void power_matrix(ll F[2][2], ll n) {
    if (n == 0 || n == 1) return;
    ll M[2][2] = {{1, 1}, {1, 0}};

    power_matrix(F, n / 2);
    multiply_matrix(F, F);

    if (n % 2 != 0)
        multiply_matrix(F, M);
}

// Returns the nth Fibonacci number modulo mod
// Complexity:  $O(\log n)$  - Efficient for n up to  $10^{18}$ 
ll fibonacci(ll n) {
    if (n == 0) return 0;
    ll F[2][2] = {{1, 1}, {1, 0}};
    power_matrix(F, n - 1);
    return F[0][0];
}

// Euler's Totient Function (Phi)
// Counts the number of integers in range [1, n] that are coprime to n.
int phi(int n) {
    int result = n;

```



```

    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}

// Precompute Phi for 1 to n (Sieve-like approach)
// Useful if you need phi for many numbers up to N.
// Complexity: O(N log(log N))
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);

    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) { // i is prime
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}

// Binary GCD (Stein's Algorithm)
// Faster than standard recursive Euclidean GCD on some hardware.
int gcd(int a, int b) {
    if (!a || !b) return a | b;

    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);

    do {
        b >>= __builtin_ctz(b);
        if (a > b) swap(a, b);
        b -= a;
    } while (b);

    return a << shift;
}

// Extended Euclidean Algorithm
// Solves: a*x + b*y = gcd(a,b)
// Returns: gcd(a,b). Updates x and y with a valid solution.
int egcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Linear Diophantine Equation: Find ANY solution
// Solves: a*x + b*y = c
// Returns: true if solution exists, updates x0, y0, and g=gcd(a,b).
bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
    g = egcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false; // No solution if c is not divisible by gcd
    }
}

```

```

// Scale the solution from egcd (which solved for =g) to =c
x0 *= c / g;
y0 *= c / g;

// Fix signs because egcd assumed positive a, b
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}

bool find_positive_solutions(int a, int b, int c, int &x, int &y, int &g) {
    if (!find_any_solution(a, b, c, x, y, g)) return false;

    double k_min = ceil(-(double)x * g / b);

    double k_max = floor((double)y * g / a);

    if (k_min <= k_max) {
        int k = (int)k_min;
        x = x + k * (b / g);
        y = y - k * (a / g);
        return true;
    }

    return false;
}

void shift_solution(int &x, int &y, int a, int b, int cnt) {
    x += cnt * b;
    y -= cnt * a;
}

// Linear Diophantine Equation: Count ALL solutions in range
// Counts solutions to a*x + b*y = c where minx <= x <= maxx and miny <= y <= maxy.
int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    if (a == 0 && b == 0) {
        if (c == 0) return (ll)(maxx - minx + 1) * (maxy - miny + 1);
        return 0;
    }
    if (a == 0 || b == 0) return 1;

    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx) shift_solution(x, y, a, b, sign_b);
    if (x > maxx) return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx) shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny) shift_solution(x, y, a, b, -sign_a);
    if (y > maxy) return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy) shift_solution(x, y, a, b, sign_a);
    int rx2 = x;

    if (lx2 > rx2) swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

```

```

    if (lx > rx) return 0;
    return (rx - lx) / abs(b) + 1;
}

// Chinese Remainder Theorem (CRT)
ll egcd_ll(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll d = egcd_ll(b, a % b, x1, y1);
    x = y1; y = x1 - y1 * (a / b);
    return d;
}

/*
 * Solves system:
 * x = a[0] (mod m[0])
 * x = a[1] (mod m[1])
 * ...
 * * @param A Vector of remainders
 * @param M Vector of moduli
 * @return pair<val, lcm> where x = val (mod lcm). Returns {-1, -1} if no solution.
 */
pair<ll, ll> chinese_remainder_theorem(const vector<ll>& A, const vector<ll>& M) {
    if (A.size() != M.size()) return {-1, -1};

    ll x = 0; // Current solution
    ll lcm = 1; // Current LCM of processed moduli

    for (size_t i = 0; i < A.size(); i++) {
        ll a = A[i];
        ll m = M[i];

        // Normalize: We want to solve x + k * lcm = a (mod m)
        // Rewrite as: k * lcm = a - x (mod m)
        // This is a linear congruence: A'k = B' (mod m)

        ll p, q;
        ll g = egcd_ll(lcm, m, p, q); // p*lcm + q*m = g

        // Check for existence: (a - x) must be divisible by gcd(lcm, m)
        if ((a - x) % g != 0) return {-1, -1};

        // Solve for k:
        // p * lcm = g (mod m) ==> multiply by (a - x)/g
        // k = p * (a - x) / g (mod m/g)

        // Use __int128 to prevent overflow during multiplication if available,
        // otherwise ensure inputs are within safe bounds.
        ll m_div_g = m / g;
        ll multiplier = (a - x) / g;

        // Safe modular multiplication for k = (p * multiplier) % m_div_g
        ll k = (p % m_div_g * (multiplier % m_div_g)) % m_div_g;
        if (k < 0) k += m_div_g;

        // Update solution x
        x += k * lcm;
        lcm *= m_div_g; // Update global LCM

        // Keep x within [0, lcm-1]
        if (x < 0) x += lcm;
    }
    return {x, lcm};
}

signed main() {
    SMTYON
    int x_x = 1;
    //cin >> x_x;
    while (x_x--) {

```

```

    return 0;
}

```

Prime factorization

math/Prime_factorization.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a)  a.begin(),a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7 , inf = 1e9 , MAXN = 1e5+5;

vector<short> spf(MAXN+5);

void sieve() {
    for(int i = 2 ; i*i <= MAXN ; i++){
        if(spf[i])continue;
        for(int j = i*i ; j <= MAXN ; j += i){
            spf[j] = i;
        }
    }
}

vector<int> getFactorization(int x) {
    vector<int> factors;
    while (x > 1) {
        int p = spf[x];
        if(!p) p = x;
        factors.push_back(p);
        x /= p;
    }
    return factors;
}

signed main() {
    SMTYON
    int x_x = 1;
    //cin >> x_x;
    while (x_x-->0) {

    }

    return 0;
}

```

geometric series

math/geometric_series.cpp

```

/*
a = the first element in the series
r = the ratio between every element and the previous ans has to be constant

to infinty:
sum = a / (1-r)
-> 0 <= r < 1.

to m:
sum = (a * (1 - r^m)) / (1-r).qZ0
*/

```

Strings (6)

Strings

2D Hash

Strings/2D_Hash.cpp

```

const int N = 2005;
const int B = 2;
const int MOD[B] = {(int) 1e9 + 7, (int) 1e9 + 9};
const int bx[B] = {29, 31};
const int by[B] = {37, 41};

int pwx[B][N], pwy[B][N];
int invx[B][N], invy[B][N];
int invBx[B], invBy[B];

#define multiHash array<int, B>

int add(int a, int b, int mod) { return (a + b) % mod; }
int sub(int a, int b, int mod) { return (a - b + mod) % mod; }
int mul(int a, int b, int mod) { return (1LL * a * b) % mod; }

int fp(int a, int b, int mod) {
    int ret = 1;
    while (b) {
        if (b & 1)
            ret = mul(ret, a, mod);
        a = mul(a, a, mod);
        b >>= 1;
    }
    return ret;
}

void pre2D() {
    for (int k = 0; k < B; k++) {
        pwx[k][0] = pwy[k][0] = invx[k][0] = invy[k][0] = 1;
        invBx[k] = fp(bx[k], MOD[k] - 2, MOD[k]);
        invBy[k] = fp(by[k], MOD[k] - 2, MOD[k]);
        for (int i = 1; i < N; i++) {
            pwx[k][i] = mul(pwx[k][i - 1], bx[k], MOD[k]);
            pwy[k][i] = mul(pwy[k][i - 1], by[k], MOD[k]);
            invx[k][i] = mul(invx[k][i - 1], invBx[k], MOD[k]);
            invy[k][i] = mul(invy[k][i - 1], invBy[k], MOD[k]);
        }
    }
}

class Hash2D {
    vector<vector<multiHash>> h;
    int n, m;

public:
    Hash2D(vector<string> &grid) {
        static bool init = []() {
            pre2D();
            return true;
        }();
        n = grid.size();
        m = grid[0].size();
        h.assign(n + 1, vector<multiHash>(m + 1));
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                for (int k = 0; k < B; k++) {
                    int val = mul(mul(grid[i - 1][j - 1] - 'a' + 1, pwx[k][i], MOD[k]), pwy[k][j], MOD[k]);
                    h[i][j][k] = add(add(sub(add(h[i - 1][j][k], h[i][j - 1][k], MOD[k]), h[i - 1][j - 1][k], MOD[k]),
                        val, MOD[k]),
                        0, MOD[k]);
                }
            }
        }
    }
}

```

```

}

multiHash get_hash(int x1, int y1, int x2, int y2) // 1-based
{
    multiHash ret;
    for (int k = 0; k < B; k++) {
        int cur = h[x2][y2][k];
        cur = sub(cur, h[x1 - 1][y2][k], MOD[k]);
        cur = sub(cur, h[x2][y1 - 1][k], MOD[k]);
        cur = add(cur, h[x1 - 1][y1 - 1][k], MOD[k]);
        // normalize
        cur = mul(cur, invx[k][x1 - 1], MOD[k]);
        cur = mul(cur, invy[k][y1 - 1], MOD[k]);
        ret[k] = cur;
    }
    return ret;
}
};

```

Hashing

Strings/Hashing.cpp

```

#include <bits/stdc++.h>
using namespace std;

/// DONOT FORGET TO CHANGE N
const int N = 4e5+5;
const int B = 2;
const int mods[3] = {1000000007, 1000000009, 998244353};
int base[B];
int pw[B][N], inv[B][N];

inline int add(int a, int b, int m) {
    int res = a+b;
    return res >= m ? res-m : res;
}
inline int sub(int a, int b, int m) {
    int res = a-b;
    return res < 0 ? res+m : res; }
inline int mul(int a, int b, int m) { return (1LL * a * b) % m; }

int fp(int b, int power, int m) {
    int res = 1;
    b %= m;
    while (power > 0) {
        if (power & 1) res = mul(res, b, m);
        b = mul(b, b, m);
        power >>= 1;
    }
    return res;
}

/// DONOT FORGET TO CALL IN MAIN
void initPows() {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

    for (int k = 0; k < B; k++) {
        uniform_int_distribution<int> dist(300, mods[k]-2);
        base[k] = dist(rng);

        pw[k][0] = inv[k][0] = 1;
        int b_inv = fp(base[k], mods[k]-2, mods[k]);

        for (int i = 1; i < N; i++) {
            pw[k][i] = mul(pw[k][i-1], base[k], mods[k]);
            inv[k][i] = mul(inv[k][i-1], b_inv, mods[k]);
        }
    }
}

struct Agent73 {

```

```

// 1-based indexing for prefixes
vector<array<int, B>> pref;

Agent73(const string& s) {
    int n = s.size();
    pref.assign(n+1, {});

    for (int i = 1; i <= n; i++) {
        for (int k = 0; k < B; k++) {
            // val * B^(i-1)
            int char_val = mul(s[i-1], pw[k][i-1], mods[k]);
            pref[i][k] = add(pref[i-1][k], char_val, mods[k]);
        }
    }
}

// l and r are 1-based
array<int, B> getHash(int l, int r) {
    array<int, B> res;
    for (int k = 0; k < B; k++) {
        // (Pref[r]-Pref[l-1]) * Inv[l-1]
        int raw_diff = sub(pref[r][k], pref[l-1][k], mods[k]);
        res[k] = mul(raw_diff, inv[k][l-1], mods[k]);
    }
    return res;
}
};

```

Kmp

Strings/Kmp.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(),a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7 , inf = 1e9 , MAXN = 1e5+5;

signed main() {
    SMTYON
    int x_x = 1;
    //cin >> x_x;
    while (x_x--) {
        string s,text;cin >> s;
        int n = s.size();

        // failer function
        //pi[i] = the size of the substring that is a prefix of s
        //and a suffix that ends in i.
        vector<int> pi(n, 0);

        for (int i = 1; i < n; i++) {
            int k = pi[i-1];
            while (k > 0 && s[i] != s[k]) k = pi[k - 1];
            if (s[i] == s[k]) k++;
            pi[i] = k;
        }

        // Finds all starting indices of 'pattern' inside 'text'
        vector<int> matches;
        int n = text.size();
        int m = s.size();
        int k = 0; // 'k' represents how many characters of the pattern we've matched so far

        for (int i = 0; i < n; i++) {
            while (k > 0 && text[i] != s[k]) k = pi[k - 1];
            if (text[i] == s[k]) k++;
            if (k == m) {
                // Calculate the 0-based starting index in the text
                matches.push_back(i-m+1);
                // To find overlapping matches, fall back to the longest valid prefix
                k = pi[k - 1];
            }
        }

        return 0;
    }
}

```

Manacher

Strings/Manacher.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(),a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7 , inf = 1e9 , MAXN = 1e5+5;

signed main() {
    SMTYON

```



```

int x_x = 1;
//cin >> x_x;
while (x_x--) {
    string tm;
    cin >> tm;
    string s = "#";
    for(auto x : tm) {
        s += x;
        s += '#';
    }
    int n = s.size();
    /*
    s = # + tm[0] + # + t[1] + # + ..... + #
    because if the palindrom has an even lenght without the # will
    be a pain, but now we can say that a # is a center of an even palindrom
    ( make it an odd palindrom with a # as a center).
    */
    vector<int> man(n);
    for(int i = 0,l=0,r=-1; i < n ;i++){
        /* if i is in the calculated range take your answer from the prevuios
        calculations NOTE:(the prevuios calcultion can make you bigger the the
        caluclated range which is not right as you do not know what after r
        that is why we take the minemum with r-i)
        else the answer is 1 as any char is palindrom.
        */
        int k = (i < r)? min(man[l+(r-i)],(r-i)): 1;
        while(i-k >= 0 && i+k < n && s[i-k] == s[i+k])k++;
        // if the current calculated range is bigger save it
        if(i+k > r){
            l = i-k;
            r = i+k;
        }
        // k-1 is the length of the palindrom in the original string (tm)
        man[i] = k-1;
    }

    /*
    if i need to the length of the ith char in the original string
    1- i is the center of an odd palindrom -> man[2*i + 1]
    2- i is the char after the center in an even palindrom -> man[2*i]
    example:
    tm = "aaaab"
    i = 2.
    man[2*i + 1] = 3 -> "_aaa_"
    man[2*i]     = 4 -> "aaa_"
    */

}

return 0;
}

```

Suffix Array

Strings/Suffix_Array.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(),a.end()
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);

void count_sort(vector<int>& p ,vector<int>& c){
    int n = p.size();
    vector<int> cnt(n),pos(n);
    for(auto x : c) cnt[x]++;
    for(int i = 1 ;i < n ;i++) pos[i] = pos[i-1]+cnt[i-1];

    vector<int> new_p(n);
    for(int i = 0 ;i < n ;i++){

```

```

    int idx = c[p[i]];
    new_p[pos[idx]] = p[i];
    pos[idx]++;
}
p = new_p;
}

signed main() {
    SMTYON

    string s;
    cin >> s;
    s += '$';
    ///! n is the size of the string + '$'
    int n = s.size();
    // IF YOU ARE GOING TO COMBINE TWO STRINGS THE SIPRATORS
    // HAS TO BE DIFF -> s + '#' + t + '$'.

    // p[i] = the index where the lexicographically smallest ith suffix starts from.
    // c[i] = which category the suffix i belongs to (equal suffixs belongs to same category)
    /*
    test:
        ababba
    p:
        6 5 0 2 4 1 3
    lcp:
        0 0 1 2 0 2 1
    */

    // Time = O(n*log^2(n))
    {
        vector<int> p(n), c(n);
        // base case
        vector<pair<char, int>> a(n);
        for(int i = 0; i < n; i++) a[i] = {s[i], i};
        sort(all(a));
        for(int i = 0; i < n; i++) p[i] = a[i].second;
        c[p[0]] = 0;
        for(int i = 1; i < n; i++) c[p[i]] = c[p[i-1]] + (a[i].first != a[i-1].first);

        // transition
        int k = 0;
        while((1 << k) < n){
            vector<pair<pair<int, int>, int>> a(n);
            for(int i = 0; i < n; i++){
                a[i] = {{c[i], c[(i+(1<<k)) % n]}, i};
            }
            sort(all(a));
            for(int i = 0; i < n; i++) p[i] = a[i].second;
            c[p[0]] = 0;
            for(int i = 1; i < n; i++) c[p[i]] = c[p[i-1]] + (a[i].first != a[i-1].first);
            k++;
        }
    }

    // Time = O(n*log(n))
    vector<int> p(n), c(n);
    {
        // base case:
        vector<pair<char, int>> a(n);
        for(int i = 0; i < n; i++) a[i] = {s[i], i};
        sort(all(a));
        for(int i = 0; i < n; i++) p[i] = a[i].second;
        c[p[0]] = 0;
        for(int i = 1; i < n; i++) c[p[i]] = c[p[i-1]] + (a[i].first != a[i-1].first);

        // iterations:
        int k = 0;
        while((1 << k) < n){
            /*
            we take binifit of the old p is already sorted
            so we add the first part of the new p to the old p

```

```

        and only sort on the first part as the second is already sorted.
        new_p[i] = (2^k new chars) + old_p[i];
    */
    // transition to new p:
    for(int i = 0 ; i < n ; i ++ ) p[i] = (p[i] - (1 << k) + n)%n;

    // sort on the first half:
    count_sort(p,c);
    vector<int> new_c(n);
    new_c[p[0]] = 0;
    for(int i = 1 ; i < n;i++){
        pair<int,int> prev = {c[p[i-1]] , c[(p[i-1] + (1 << k))%n]};
        pair<int,int> cur = {c[p[i]] , c[(p[i] + (1 << k))%n]};
        new_c[p[i]] = new_c[p[i-1]] + (prev != cur);
    }
    c = new_c;
    k++;
}

}

// longest common prefix LCP: O(n)
vector<int> lcp(n);
int k = 0;
for(int i = 0 ; i < n-1 ; i++){
    int idx = c[i];
    if(idx == 0) continue;
    int j = p[idx-1];
    while(s[i+k] == s[j+k]) k++;
    lcp[idx] = k;
    k = max(0 , k-1);
}
/*
lcp[i] = lcp[ s[pos[i],.....n-1] , s[pos[i-1],.....n-1] ].
to find the lcp between suffix starts at i and a suffix starts at j:
min(lcp[i+1] , lcp[i+2], ..... , lcp[j]).
this is normal range query which you can use segment tree or sparse table.
*/
}

```

Z Algo

Strings/Z_Algo.cpp

```

#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(),a.end()
#define ll long long
#define ld long double
#define endl '\n'
#define SMTYON ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
const ll mod = 1e9+7 , inf = 1e9 , MAXN = 1e5+5;

signed main() {
    SMTYON
    int x_x = 1;
    //cin >> x_x;
    while (x_x--) {
        // the size of the substring that is a prefix of s[0,n-1] and also a prefix of s[i,n-1].
        string s;cin >> s;
        int n = s.size();
        // z[0] = 0 -> will not change any thing in the logic
        vector<int> z(n);
        for(int i=1,l=0,r=-1; i < n ;i++){
            /* if i is in the range that we allready explored then use the old calculations.
               else start from the beginning with the naiv solution.
            */
            int k = (i<r)? min(z[(i-l)],(r-i)):0;
            while(i+k < n && s[k] == s[i+k]) k++;
            // if we explored a bigger rang store it.
            if(i+k > r){
                l = i;
                r = i+k;
            }
            z[i] = k;
        }
    }

    return 0;
}

```

Other (6)

other

Count Inversions

other/Count_Inversions.cpp

```
// Count Inversions in n*log(n).

// This function merges two sorted subarrays arr[l..m] and arr[m+1..r]
// and also counts inversions in the whole subarray arr[l..r]
int countAndMerge(vector<int>& arr, int l, int m, int r) {

    int n1 = m-l+1, n2 = r-m;

    vector<int> left(n1), right(n2);
    for (int i = 0; i < n1; i++) left[i] = arr[i+l];
    for (int j = 0; j < n2; j++) right[j] = arr[m+1+j];

    int res = 0;
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (left[i] <= right[j]) arr[k++] = left[i++];
        else {
            arr[k++] = right[j++];
            res += (n1 - i);
        }
    }

    while (i < n1) arr[k++] = left[i++];
    while (j < n2) arr[k++] = right[j++];
    return res;
}

int countInv(vector<int>& arr, int l, int r){
    int res = 0;
    if (l < r) {
        int m = (r+l)/2;

        res += countInv(arr, l, m);
        res += countInv(arr, m + 1, r);

        res += countAndMerge(arr, l, m, r);
    }
    return res;
}

int inversionCount(vector<int> &arr) {
    int n = arr.size();
    return countInv(arr, 0, n-1);
}
```

Multiset Hashing

other/Multiset_Hashing.cpp

```

const int mod = 1e9 + 7;
const int B = 2;
int r[B];
#define multiHash array<int, B>
int add(int a, int b) { return (a % mod + b % mod + mod) % mod; }
int mul(int a, int b) { return (a % mod * b % mod) % mod; }
int fp(int b, int e)
{
    if (!e)
        return 1;
    int ret = fp(b, e >> 1);
    ret = mul(ret, ret);
    return (e & 1 ? mul(ret, b) : ret);
}

void pre()
{
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    for (int i = 0; i < B; ++i)
        r[i] = rng() + 1;
}

class MultisetHash
{
    vector<multiHash> h;
    vector<int> v;
    int n;

public:
    MultisetHash(vector<int> &v)
    {
        static bool init = []()
        {
            pre();
            return true;
        }();
        this->v = v;
        n = v.size();
        h.resize(n + 1);
        h[0][0] = h[0][1] = 1;
        for (int i = 1; i <= n; ++i)
        {
            int cur = v[i - 1];
            for (int j = 0; j < B; ++j)
            {
                h[i][j] = mul(h[i - 1][j], add(cur, r[j]));
            }
        }
    }

    int get_hash(int l, int r) // 1-based
    {
        multiHash ret;
        for (int i = 0; i < B; ++i)
        {
            ret[i] = mul(h[r][i], fp(h[l - 1][i], mod - 2));
        }
        return (ret[0] | (ret[1] << 32));
    }
};

```

Ordered set

other/Ordered_set.cpp

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

#define ordered_set tree<int, null_type,less<int>, rb_tree_tag,tree_order_statistics_node_update>

template <class type1>
using ordered_multiset = tree <type1, null_type, less_equal <type1>, rb_tree_tag, tree_order_statistics_node_update>;
```

Pragma

other/Pragma.cpp

```
//#pragma GCC optimize("Ofast")
//#pragma GCC target("avx,avx2,fma")
//#pragma GCC optimize("unroll-loops")
```

Random

other/Random.cpp

```
// the engine
//int
mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());

//unsigned long long
mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());

//(between 0 and 2^32 - 1)
rng()

// Generates a random integer between 1 and 6
uniform_int_distribution<int> rng_rand(1, 6);
rng_rand(rng);

/// Generates a random double between 0.0 and 1.0
uniform_real_distribution<double> real_rand(0.0, 1.0);
real_rand(rng);
```

Super FIO

other/Super_FIO.cpp

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
const int BUF_SZ = 1 << 15;

// BeginCodeSnip{Input}
inline namespace Input {
char buf[BUF_SZ];
int pos;
int len;
char next_char() {
    if (pos == len) {
        pos = 0;
        len = (int)fread(buf, 1, BUF_SZ, stdin);
        if (!len) { return EOF; }
    }
    return buf[pos++];
}

int read_int() {
    int x;
    char ch;
    int sgn = 1;
```

```

    while (!isdigit(ch = next_char())) {
        if (ch == '-') { sgn *= -1; }
    }
    x = ch - '0';
    while (isdigit(ch = next_char())) { x = x * 10 + (ch - '0'); }
    return x * sgn;
}
} // namespace Input
// EndCodeSnip
// BeginCodeSnip{Output}
inline namespace Output {
char buf[BUF_SZ];
int pos;

void flush_out() {
    fwrite(buf, 1, pos, stdout);
    pos = 0;
}

void write_char(char c) {
    if (pos == BUF_SZ) { flush_out(); }
    buf[pos++] = c;
}

void write_int(int x) {
    static char num_buf[100];
    if (x < 0) {
        write_char('-');
        x *= -1;
    }
    int len = 0;
    for (; x >= 10; x /= 10) { num_buf[len++] = (char)('0' + (x % 10)); }
    write_char((char)('0' + x));
    while (len) { write_char(num_buf[--len]); }
    write_char('\n');
}

// auto-flush output when program exits
void init_output() { assert(atexit(flush_out) == 0); }
} // namespace Output
// EndCodeSnip

int main() {
    init_output();
    int M = read_int();
    int N = read_int();
    int ans = 0;
    for (int i = 0; i < N; i++) {
        ans = (ans + read_int()) % MOD;
        if (M == 1) { write_int(ans); }
    }
    if (M == 0) { write_int(ans); }
}

```